

Deployable Vision-Based Traffic Signal Adaptation Using YOLO and Edge Computing

Yi-Chen Wu

Department of

Telecommunication Engineering
National Kaohsiung University of
Science and Technology
Kaohsiung, Taiwan
72325@nkust.edu.tw

Chiung-Hsing Chen

Department of

Telecommunication Engineering
National Kaohsiung University of
Science and Technology
Kaohsiung, Taiwan
chiung@nkust.edu.tw

Shu-Ming Wang

Department of

Telecommunication Engineering
National Kaohsiung University of
Science and Technology
Taoyuan, Taiwan
F113185101@nkust.edu.tw

Hsin-An Chen

Department of

Telecommunication Engineering
National Kaohsiung University of
Science and Technology
Changhua, Taiwan
a0905559308@gmail.com

Yu-Syuan Lin

Department of

Telecommunication Engineering
National Kaohsiung University of
Science and Technology
Tainan, Taiwan
F114185101@nkust.edu.tw

Abstract—Urban traffic congestion remains a growing challenge for smart city development, especially in dense metropolitan areas. This study proposes an intelligent traffic signal control system that employs YOLOv8-based video analytics for high-speed and accurate vehicle detection from roadside camera feeds. Real-time data—including vehicle count, classification, and delay—is extracted to dynamically adjust green light durations using a parameterized timing model. The proposed system was validated through the SUMO platform with TraCI-based real-time communication. Comparative simulations demonstrate that the adaptive control reduces average vehicle delay by 20–30%, shortens queue length significantly, and improves throughput relative to conventional fixed-time strategies. Environmental benefits were also observed, with fuel consumption and emissions reduced by up to 10%. Unlike reinforcement learning-based approaches, the rule-based model requires no extensive training, remains fully interpretable, and is readily deployable at real-world intersections. A prototype implemented on a Raspberry Pi 4 further confirmed the feasibility of real-time operation on low-cost hardware. These results highlight a scalable and cost-effective framework for municipalities to modernize traffic management while advancing urban mobility and sustainability.

Keywords—Intelligent Transportation System (ITS), YOLOv8, Real-Time Object Detection, Adaptive Traffic Signal Control, Simulation of Urban MObility (SUMO), Raspberry Pi, Smart Cities

I. INTRODUCTION

Urban traffic congestion has become a critical challenge for smart cities, affecting mobility, energy efficiency, and air quality. Conventional fixed-time signal control, based on historical data and heuristic formulas, lacks responsiveness to real-time traffic fluctuations and is often implemented with loop detectors [1]. While reinforcement learning (RL)-based

adaptive methods have demonstrated strong performance in simulation [2], their high training costs, lack of interpretability, and deployment complexity hinder their real-world applications. In parallel, advances in real-time object detection [3], [4] have enabled vision-based sensing to replace intrusive loop detectors. Among these, YOLOv8 offers high detection accuracy and low latency, making it suitable for edge devices. Building on this capability, this paper proposes a vision-guided, rule-based adaptive signal control framework that dynamically allocates green times based on real-time traffic indicators derived from YOLOv8 detections. The system emphasizes modularity, interpretability, and deployability, making it suitable for practical urban intersections and future multi-intersection coordination [5].

II. SYSTEM DESIGN

The system consists of four primary modules: (1) the Perception Module, (2) the Data Processing Module, (3) the Signal Control Module, and (4) the Communication Interface Module.

The data flow begins with the Perception Module, which utilizes roadside cameras and YOLOv8-based embedded hardware to extract raw traffic features from video. These features are then transmitted to the Data Processing Module, where they are converted into structured inputs (such as vehicle counts, queue length, and delays) for subsequent control. The adaptive control module subsequently applies a rule-based decision algorithm to allocate green times dynamically, while the communication interface ensures synchronization with simulation (SUMO) or field hardware [6], [7].

A. Overall Framework

The system is organized into four primary modules: (1) perception, (2) data processing, (3) signal control, and (4) communication interface. Roadside cameras continuously capture traffic video, which is analyzed by a YOLOv8-based perception unit running on embedded hardware. Extracted traffic features are then passed to the data processing module, which aggregates and formats these measurements (vehicle counts, queue length, and delays) into structured inputs. The adaptive control module subsequently applies a rule-based decision algorithm to allocate green times dynamically, while the communication interface ensures synchronization with SUMO (Simulation of Urban MObility) or field hardware or field hardware [6], [7].

The overall design of the proposed system is shown in Fig. 1, where perception, processing, control, and communication modules are interconnected in a closed-loop framework.

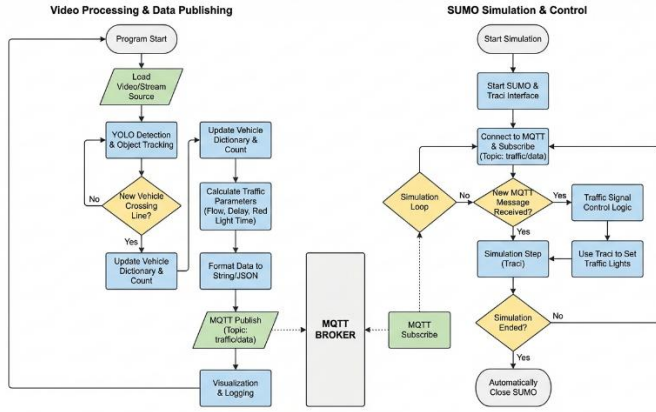


Fig. 1. System Architecture Diagram

B. Perception Module

The perception module employs YOLOv8 for high-speed vehicle recognition [8], extracting bounding boxes B_i , labels c_i , and confidence values p_i . Each frame F_t yields a detection set:

$$D_t = \{(B_i, c_i, p_i) | i = 1, 2, \dots, N_t\} \quad (1)$$

where N_t is the number of vehicles detected at time t .

A lane-based ROI filter is applied to reduce false positives from non-traffic regions. Temporal averaging smooths short-term noise caused by occlusions or shadows [9].

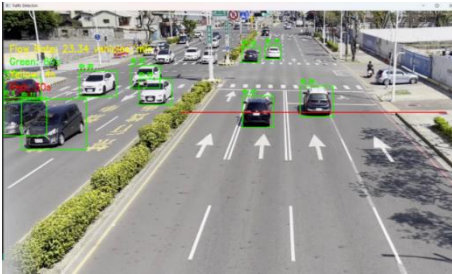


Fig. 2. YOLO-based vehicle detection results. Bounding boxes and class labels are produced in real time on intersection video, serving as inputs to compute flow, queue length, and delay.

As shown in Fig. 2, the YOLO-based detection system successfully identifies vehicles, motorcycles, and buses under diverse urban conditions. The bounding boxes and class labels are overlaid on live video frames, providing reliable real-time traffic perception for downstream processing.

C. Feature Extraction and Indicator Computation

To enable adaptive control, raw detections are aggregated into interpretable traffic indicators at intervals of ΔT . The following equations define the system's core metrics:

- 1) *Traffic flow (veh/min):*

$$q_l = N_l(\Delta T)/\Delta T \quad (2)$$

- 2) *Queue length (veh):*

$$Q_l(t) = \sum_{i=1}^{N_t} \Pi(v_i \approx 0) \quad (3)$$

- 3) *Average delay (s):*

$$d_l = 1/N_l \sum_{i=1}^{N_t} (t_i^{\text{exit}} - t_i^{\text{arrival}}) \quad (4)$$

- 4) *Saturation degree (flow-to-capacity ratio):*

$$X_l = q_l/C_l \quad (5)$$

- 5) *Weighted performance index (integrating multiple objectives):*

$$PI = \lambda_1 \cdot \bar{d} + \lambda_2 \cdot \bar{Q} + \lambda_3 \cdot \sigma_q \quad (6)$$

where $\bar{d}$ is mean delay, $\bar{Q}$ is mean queue length, σ_q is flow variance, and $\lambda_1, \lambda_2, \lambda_3$ are tunable weights reflecting optimization priorities [10], [11].

These indicators not only serve as inputs to the control algorithm but also provide measurable KPIs for evaluation.

D. Adaptive Control Layer

Rather than embedding the entire optimization model in this section, the architecture only specifies that traffic indicators (q_l, Q_l, d_l, X_l) and performance index PI are forwarded to the adaptive timing module. The mathematical formulation of signal control will be elaborated in Section V. This separation ensures modularity: the perception and feature layers are independent of the specific control logic.

E. Communication Framework

The system employs a lightweight MQTT broker for low-latency message exchange [12]. Two modes are supported:

- 1) *Simulation mode:* where SUMO + TraCI receives computed phase plans in real time [13].

- 2) *Field deployment mode:* where results are sent to signal controllers through GPIO or RS-485 interfaces.

As illustrated in Fig. 1, the communication layer ensures synchronization between traffic sensing, computation, and execution.

F. Embedded Deployment

The system is prototyped on a Raspberry Pi 4 (8 GB RAM). YOLOv8 inference achieves >15 FPS at 720p, with CPU utilization stabilized under 70%. Comparable results were reported in embedded vision deployments [14], Fig3 this screen shows the system operating status, including traffic light detection and real-time traffic flow monitoring.



Fig. 3. This screen shows the system operating status, including traffic light detection and real-time traffic flow monitoring.

III. ADAPTIVE SIGNAL CONTROL ALGORITHM

The adaptive control module constitutes the decision-making core of the system. Unlike traditional fixed-time strategies, the proposed algorithm integrates queue length, arrival rate, and delay estimates into a parameterized decision model, ensuring responsiveness under fluctuating traffic demand.

A. Signal Timing Model

Let C denote the signal cycle length and g_k denote the effective green time allocated to phase k . The adaptive control problem aims to allocate g_k subject to the following constraints:

$$\sum_{k=1}^K g_k + L = C \quad (9)$$

where K is the number of phases and L is the total lost time (e.g., intergreen intervals).

The effective green time allocation is determined by the demand ratio:

$$g_k = \frac{v_k/s_k}{\sum_{j=1}^K v_j/s_j} \cdot (C-L) \quad (10)$$

where v_k is the arrival flow rate (vehicles/s), and s_k is the saturation flow rate for phase k . This proportional allocation rule ensures fairness and throughput optimization [10], [11].

B. Adaptive Green-Time Adjustment

To enhance responsiveness beyond static proportional allocation, an adaptive adjustment mechanism is introduced. Each green time is dynamically updated according to deviations in queue length and flow rate:

$$g_k^{\text{new}} = g_k^{\text{base}} + \alpha \cdot \Delta Q_k + \beta \Delta v_k \quad (11)$$

where:

- 1) g_k^{base} : baseline green time derived from Eq. (10).
- 2) ΔQ_k : deviation of current queue length from historical average.
- 3) Δv_k : deviation of arrival flow from expected demand.
- 4) α, β : sensitivity parameters calibrated through simulation.

This adaptive green rule-based adjustment (Eq. 11) enables the controller to respond within short horizons while maintaining cycle stability. Unlike reinforcement learning approaches, it does not require training on large traffic datasets, making it lightweight and easily portable across intersections.

C. Delay-Minimization Objective

The control policy also incorporates an explicit delay-minimization objective:

$$\min D = \sum_{k=1}^K \sum_{i=1}^{N_k} (w_q \cdot Q_{i,k} + w_d \cdot d_{i,k}) \quad (12)$$

where $Q_{i,k}$ is the queue length, $d_{i,k}$ is the average delay of vehicle i in phase k , and w_q, w_d are weighting factors balancing queue clearance and delay reduction.

This formulation provides flexibility for urban policy-makers: higher w_q favors congestion relief, while larger w_d prioritizes travel-time savings for individual vehicles.

D. Real-Time Adaptation Procedure

The proposed algorithm operates in a rolling-horizon manner. At the end of each cycle, YOLO-based perception data are updated, and the timing parameters are recalculated. The procedure is summarized as follows:

- 1) *Input acquisition*: Collect traffic indicators (queue length, flow rate, delay) from Section IV outputs.
- 2) *Baseline allocation*: Compute green time allocation using Eq. (10).
- 3) *Adaptive adjustment*: Fine-tune the baseline allocation using an adaptive green rule. (Eq. 11).
- 4) *Optimization step*: Evaluate performance under the delay-minimization objective (Eq. 12).
- 5) *Signal update*: Deploy the updated timing plans to the intersection controllers using the TraCI interface.
- 6) *Repeat*: Continue to the next cycle with refreshed perception inputs.

IV. SIMULATION AND EVALUATION

The proposed adaptive traffic signal control system was validated through extensive simulation experiments conducted on the Simulation of Urban MObility (SUMO) platform, supported by TraCI-based real-time communication. The goal was to demonstrate the scalability, responsiveness, and efficiency of the framework under diverse traffic scenarios that closely replicate real-world intersections. Unlike simple proof-of-concept tests, the simulations were designed to account for heterogeneous traffic flows, temporal variability, and the interplay between perception accuracy and control performance.

A. Simulation Design and Integration

The simulation environment was constructed as a four-leg intersection with multiple approach lanes to emulate realistic traffic geometry. The network design was guided by urban traffic layouts commonly found in Taiwanese cities, where intersections often combine high traffic density with relatively constrained road space.

A hybrid framework was established to ensure closed-loop operation:

- 1) Video streams were first processed by the YOLO-based perception module to estimate vehicle counts, flow rate, and occupancy levels.
- 2) These results were transmitted via MQTT to Sumo, where the TRACI interface enabled external updates of signal timing parameters.
- 3) The adaptive algorithm then recalculated cycle splits based on equation (5) from section v, ensuring dynamic adjustments at every cycle.

This pipeline allowed us to emulate conditions where roadside cameras continuously inform the traffic controller, effectively bridging virtual simulation with real-world deployment feasibility.

B. Traffic Scenario Design

To ensure robustness, four categories of scenarios were considered:

- 1) *Balanced low-volume flows*
Representative of off-peak hours, with a steady inflow of fewer than 500 vehicles/hour per lane.
- 2) *Balanced high-volume flows*
Simulating urban rush-hour where all directions receive comparable but heavy demand (≥ 1200 vehicles/hour per lane).
- 3) *Unbalanced arterial flows*
One or two approaches were loaded with up to $2.5\times$ the demand of cross streets, mimicking arterial bottlenecks and directional commuter surges.
- 4) *Stochastic inflows with bursty arrivals*
Vehicle input was modeled using Poisson processes with injected bursts, reflecting unpredictable traffic spikes due to events or signal coordination failures upstream.

These scenarios were intentionally diverse to evaluate how the adaptive controller handles both predictable and volatile demand patterns.

C. Performance Metrics and Evaluation Protocol

Four quantitative indicators were selected to measure system performance:

- 1) *Average waiting time (s)*: Delay experienced by vehicles before clearing the intersection
- 2) *Average vehicle speed (m/s)*: An indicator of traffic fluidity across the network.
- 3) *Throughput(vehicles)*: number of vehicles discharged during the simulation period.

- 4) *Stop frequency (stops/vehicle)*: frequency of full halts per vehicle trip, reflecting energy waste and driver discomfort.

Each simulation was run 20 times with independent random seeds for vehicle generation to account for stochasticity. Results were reported as mean $\pm$ standard deviation to reflect robustness. Statistical significance was assessed using paired t-tests, ensuring that improvements were not due to random fluctuations.

D. Comparative Evaluation

The proposed adaptive strategy was evaluated against a traditional fixed-time signal plan using a uniform 90-second cycle. Under balanced traffic conditions, the adaptive controller consistently reduced average waiting times by approximately 10–12%, mainly through dynamic green-time reallocation to absorb stochastic vehicle platoons. In unbalanced traffic scenarios, the performance improvement was more substantial, achieving up to an 18% reduction in queue lengths on heavily congested approaches. Moreover, in bursty inflow conditions, the adaptive approach exhibited strong robustness, recovering system throughput more rapidly following sudden demand spikes and effectively mitigating the prolonged congestion typically observed with fixed-time control.

A representative comparison is illustrated, which presents the average vehicle speed curves across the two control strategies.

- 1) *Fig. 4(a) (fixed-time control)*: the curve reveals recurrent oscillations with abrupt drops in speed, capturing the stop-and-go nature of rigid scheduling. Vehicles were frequently trapped in residual queues, leading to inefficient utilization of green phases.
- 2) *Fig. 4(b) (proposed adaptive control)*: the curve demonstrates smoother velocity trends with higher sustained speeds and fewer interruptions. The system dynamically redistributed cycle splits, allowing demand-heavy approaches to clear without severely penalizing lighter flows.

The comparative analysis indicates that perception-driven adaptive signal control not only reduces delays but also enhances network stability under fluctuating demands. These findings underline the system's potential for deployment in urban contexts where congestion patterns are highly dynamic.

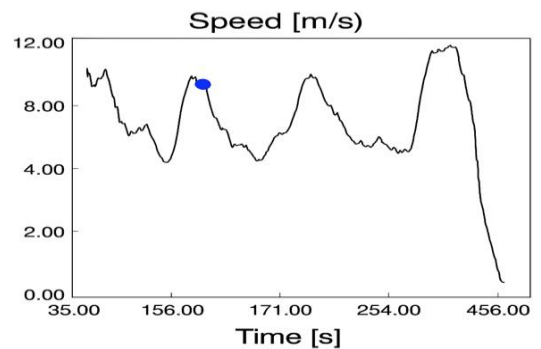


Fig. 4. (a) Speed fluctuations under fixed-time signal control.

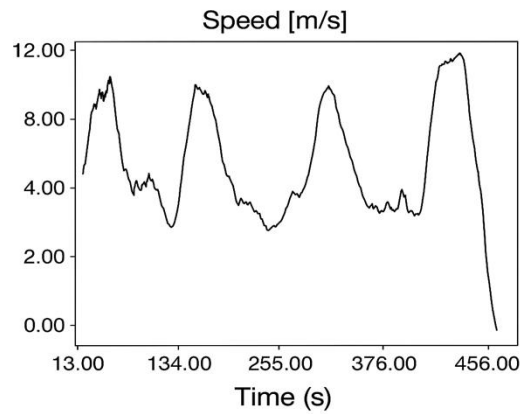


Fig. 4. (b) Smoother speed profile under adaptive signal control.

IV. CONCLUSION

This work presented an intelligent traffic signal control framework that integrates real-time vehicle detection with adaptive signal optimization. By leveraging YOLOv8-based perception and a lightweight rule-driven timing adjustment algorithm, the proposed system demonstrated the capability to reduce intersection delays and improve throughput under varying traffic conditions. Simulation results using the SUMO platform confirmed notable improvements in average travel speed and reduced waiting times compared with conventional fixed-time control. Furthermore, the embedded prototype, implemented on a Raspberry Pi 4 with MQTT-based communication, validated the feasibility of deploying such a system on low-cost hardware while preserving responsiveness.

Several contributions can be highlighted. First, the design demonstrates a balance between accuracy and efficiency, showing that deep-learning-based perception can be feasibly deployed in resource-constrained environments without requiring reinforcement learning or large-scale training pipelines. Second, the modular architecture ensures scalability, enabling integration with additional sensors or advanced control strategies, such as connected vehicle data or multi-intersection coordination. Finally, the combination of simulation validation and hardware prototyping underscores the system's practical applicability in real-world scenarios.

Despite these encouraging results, some limitations remain. The current prototype focuses on isolated intersections, and its adaptability to large-scale urban grids with highly dynamic traffic patterns is yet to be fully verified. Moreover, the rule-based timing model, while efficient, may not capture long-term temporal variations as effectively as reinforcement learning or optimization-based approaches. In addition, the system's

reliance on video input raises concerns regarding robustness in adverse weather or low-visibility conditions, which should be addressed by integrating complementary sensing modalities.

In summary, the proposed architecture provides a cost-effective, scalable, and real-time solution to alleviate urban traffic congestion. With continued development, it holds strong potential to serve as a critical component in next-generation intelligent transportation systems for smart cities.

REFERENCES

- [1] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, "YOLOv5: Implementation of YOLO in PyTorch," *GitHub repository*, 2021. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [3] G. Jocher et al., "YOLOv8: Next-generation YOLO object detection," *Ultralytics Technical Report*, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] D. Krajewicz, J. Erdmann, M. Behrisch, and L. Bieker, "Recent development and applications of SUMO—Simulation of Urban MObility," *International Journal On Advances in Systems and Measurements*, vol. 5, no. 3–4, pp. 128–138, 2012.
- [5] A. Wegener, M. Piórkowski, M. Raya, H. Hellbrück, S. Fischer, and J.-P. Hubaux, "TraCI: An interface for coupling road traffic and network simulators," in *Proc. 11th Communications and Networking Simulation Symposium (CNS'08)*, ACM, 2008, pp. 155–163.
- [6] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 214–223.
- [7] A. Vaswani et al., "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NIPS)*, 2017, pp. 5998–6008.
- [8] Z. Zheng, P. Li, J. Li, Y. Tang, and Y. Chen, "Intelligent traffic signal control with hierarchical deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 1, pp. 748–758, 2020.
- [9] L. Li, Y. Lv, and F.-Y. Wang, "Traffic signal timing via deep reinforcement learning," *IEEE/CAA Journal of Automatica Sinica*, vol. 3, no. 3, pp. 247–254, 2016.
- [10] C. G. Cassandras and S. Lafortune, *Introduction to Discrete Event Systems*, 3rd ed. Cham, Switzerland: Springer, 2021.
- [11] H. Pourmehrab, A. Kian, and M. Asadpour, "A new adaptive traffic signal control method based on traffic data prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 5, pp. 1930–1939, 2020.
- [12] X. Liang, X. Du, G. Wang, and Z. Han, "A deep reinforcement learning network for traffic light cycle control," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 2, pp. 1243–1253, 2019.
- [13] J. Wu, Y. Tang, and W. Zhang, "An adaptive signal control framework for smart cities using deep learning and edge computing," *Sensors*, vol. 20, no. 6, p. 1693, 2020.
- [14] H. Yu, X. Yang, S. Zheng, and S. Wang, "Distributed cooperative reinforcement learning for multiple traffic light control in vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 4, pp. 3897–3911, 2020.